

COP2800C Module 4 Graded Programming Assignment

In this assignment, we will practice working with while loops to process data dynamically by reading and analyzing the Palmer Penguins dataset. This exercise reads an input file line by line and uses accumulators and control structures to implement a simple data analysis task. For this assignment you will be completed the code in the starter file by locating the TODO markers and adding the required code.

- **DO NOT REMOVE THE TODO marker comments, I use these for grading**

Background:

An **accumulator** is a variable used to maintain a cumulative total or aggregated result as a loop iterates through a dataset. Each iteration of the loop updates the accumulator by adding, subtracting, or combining values based on specific conditions. For example, when processing a list of numbers, an accumulator can be used to calculate the sum or product of all the elements. In decision-based scenarios, such as counting occurrences of specific items, the accumulator increments when a condition is met. Accumulators are initialized before the loop begins, typically set to a neutral value such as 0 for sums or counts and 1 for products. By the end of the loop, the accumulator contains the desired aggregated result.

Here's a simple example of a sum accumulator in a while loop using pseudocode:

```
# declare and initialize variables
integer number = 0    # input variable
integer sum = 0       # accumulator, starts at 0

# read positive numbers until 0 or negative value is entered
number = get next input value
while number > 0 {
    # add the current number to the accumulator
    sum = sum + number
    # get another value
    number = get next input value
}

# Print the result
print("The total sum is:", sum)
```

In this lab a **CSVReader** class is included in the starter file which includes a **readFile** method. This method is called repeatedly in a while loop to read one row at a time from the PalmerPenguins CSV (comma-separated value) file. Branching logic within the loop will check whether a line contains a

penguin species name and increment the appropriate accumulator variables to maintain a running total of the occurrences of different penguin species. After the loop exits, the totals for each species are displayed.

Instructions

Note: The `PalmerPenguins.csv` file is required to complete this assignment. It must reside in the same location as your source file. The file can be found in the GitHub Classroom repository with the `PalmerPenguinsM4.java` starter file.

1. Update the 4-line ID header at the top of the file, reflecting the new file name, submission date, and summary.
2. Note the changes present in the starter file. The code which introduces the penguin species has been removed. We have met the penguins, now we can proceed with more analysis of the research data. The "count" constants have also been removed; the counts will now be determined dynamically based on the data. We only need to retain the species name constants (the strings) for now.
3. Add the new functionality:
 - Review the `CSVReader` class included at the bottom of the starter file. Note the JavaDoc comments `/** */` which allow the javadoc utility to produce API-formatted documentation about the method's behavior.
 - In the main method of the `PalmerPenguins4` class, find the TODO 1 marker. Declare 4 int variables after that line: one variable to track the current row that is being read from the file, and 3 accumulator variables to track the total number of each species. Initialize all variables to 0 in their declaration statements, e.g.,

`int currRow = 0`

- Find the TODO 2 marker and complete the if else branching statements to check for the remaining species and increment the corresponding accumulator variables.
- Find the TODO 3 marker. Now that the while loop has exited, the final accumulator values can be displayed; insert 3 separate `println` statements to print their values so the output matches the Expected Output shown here:

Expected Output

found headers:

`SampleNumber,Species,CulmenLength_mm,CulmenDepth_mm,FlipperLength_mm,BodyMass_g,Sex`
`Chinstrap count = 68`
`Gentoo count = 123`
`Adelie count = 151`

4. Test Your Program

- Run your program to verify the output.

5. Screenshot Your Output

- Take a screenshot of your program's output in the "Run I/O" pane of your IDE (e.g., jGRASP). Do not screenshot source code.

Be sure to maintain clean, readable code which follows our course coding conventions:

- Use 3-4 spaces for each level of indentation; do not use tabs.
- Separate logical sections with a blank line.
- Keep lines under 80 characters long, including comments.
- Include descriptive comments.

Submission

1. Submit the PalmerPenguinsM4.java file to your GitHub repository.
 - Do **not** include .class files.
 - Ensure your code adheres to our coding conventions.
2. Upload Your Screenshot to the repository.